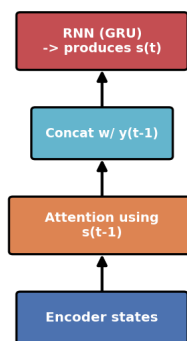


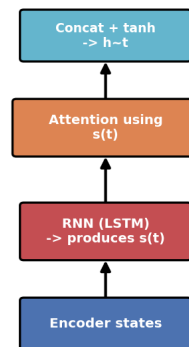
Bahdanau Attention vs Luong Attention

*Architecture, Neural Network Implementation & Key Differences Explained
Visually*

Bahdanau (Additive)



Luong (Multiplicative)



Covers: Recap of attention • Bahdanau architecture • Luong architecture • Neural network implementation details • Score functions • Key differences & when to use which

1. Quick Recap: What Is Attention?

Attention lets a decoder "look back" at every encoder hidden state, instead of relying on a single fixed-length context vector. At each decoding step, the model computes a set of **alignment scores** (one per encoder position), turns them into weights via softmax, and takes a weighted sum of encoder states to form a **context vector** $c(t)$.

The two most influential early formulations of this idea are **Bahdanau Attention** (2014) and **Luong Attention** (2015). Both compute a context vector the same general way, but differ in **when** attention is applied in the decoding step, **what** hidden state is used to compute it, and **how** the alignment scores are calculated.

2. The Core Difference in One Picture

Bahdanau: Attention BEFORE the RNN step



Luong: Attention AFTER the RNN step



Figure 1: Bahdanau computes attention BEFORE the RNN step (using $s(t-1)$); Luong computes it AFTER (using $s(t)$).

3. Bahdanau Attention - Architecture

Introduced by Bahdanau, Cho & Bengio (2014) in "Neural Machine Translation by Jointly Learning to Align and Translate", this was the original attention mechanism. It's also called **additive attention**, because of how its alignment score is computed.

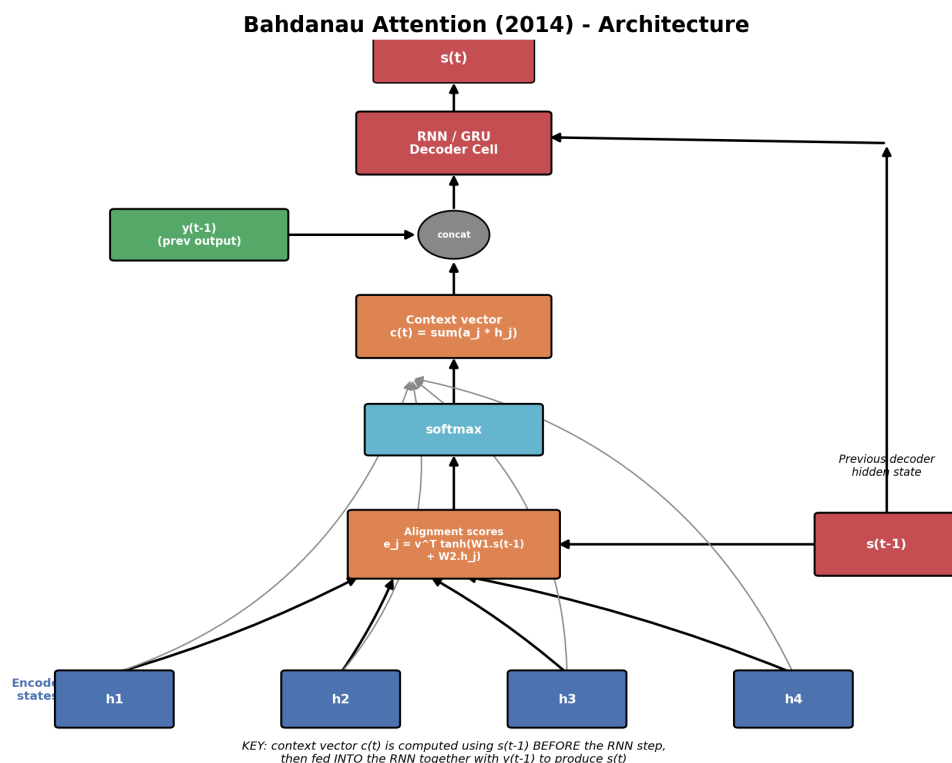


Figure 2: Full Bahdanau attention flow - attention runs BEFORE the decoder RNN cell, using the previous hidden state $s(t-1)$.

How it works, step by step:

- 1. **Alignment scores:** using the *previous* decoder hidden state $s(t-1)$ and every encoder hidden state h_j , compute a score $e_j = v^T \tanh(W1.s(t-1) + W2.h_j)$.
- 2. **Softmax:** normalize the scores into attention weights a_j that sum to 1.
- 3. **Context vector:** compute $c(t)$ as the weighted sum of encoder states: $c(t) = \sum(a_j * h_j)$.
- 4. **Feed into RNN:** concatenate $c(t)$ with the previous output embedding $y(t-1)$, and feed this combined vector INTO the RNN cell to produce the new hidden state $s(t)$.

Because attention uses $s(t-1)$ - the state from BEFORE this decoding step - context is computed first, then folded into the RNN's input for this step. The original paper also used a single-layer **bidirectional GRU** for the encoder.

4. Luong Attention - Architecture

Introduced by Luong, Pham & Manning (2015) in "*Effective Approaches to Attention-based Neural Machine Translation*", this design simplified and generalized Bahdanau's approach. It's also called **multiplicative attention** in its simplest (dot-product) form.

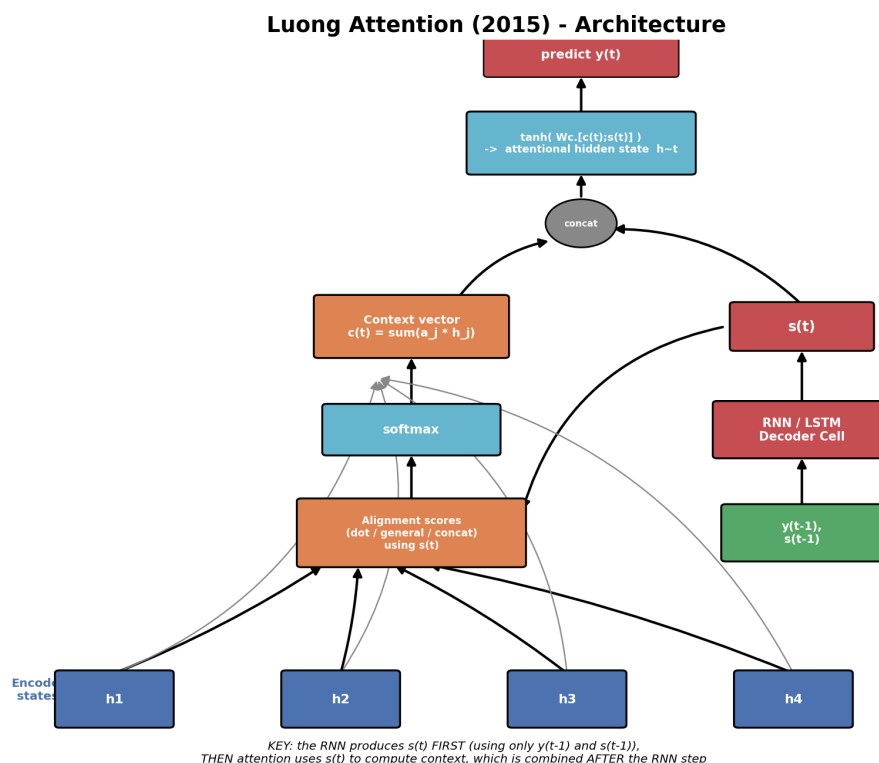


Figure 3: Full Luong attention flow - the decoder RNN runs FIRST, then attention uses the resulting current hidden state $s(t)$.

How it works, step by step:

- 1. **RNN step first:** the decoder RNN (typically a stacked LSTM) processes $y(t-1)$ and $s(t-1)$ directly - with NO attention involved yet - producing the current hidden state $s(t)$.
- 2. **Alignment scores:** using this *current* hidden state $s(t)$ and every encoder state h_j , compute alignment scores via one of three score functions (dot, general, or concat).
- 3. **Softmax + Context vector:** exactly as before - normalize into weights a_j , then compute $c(t) = \sum(a_j * h_j)$.
- 4. **Combine after the RNN:** concatenate $c(t)$ with $s(t)$, pass through a tanh layer to produce an **attentional hidden state** $h_{\sim t} = \tanh(Wc.[c(t); s(t)])$, which is then used to predict $y(t)$.

Because attention uses $s(t)$ - the CURRENT state - context is computed after the RNN step and blended in afterward, right before making the prediction. Luong's paper also proposed simpler score functions that avoid Bahdanau's extra feed-forward layer.

5. Neural Network Implementation

In terms of actual trainable layers, both mechanisms are implemented with small feed-forward / linear layers plus a softmax - no recurrence of their own. Here's what each requires:

Bahdanau (Additive) Implementation

Requires **three** learned weight matrices/vectors: W1 (applied to the decoder state), W2 (applied to encoder states), and v (collapses the combined vector down to a single score). A tanh non-linearity sits between the linear combination and the final scoring.

$$e_j = v^T \cdot \tanh(W1 \cdot s(t-1) + W2 \cdot h_j)$$

In code (conceptually, using Keras/PyTorch-style layers):

```
W1 = Dense(units) # applied to decoder hidden state
W2 = Dense(units) # applied to encoder outputs
V = Dense(1) # scores each combined vector down to 1 number
score = V(tanh(W1(s_prev) + W2(enc_outputs)))
attention_weights = softmax(score, axis=1)
context = sum(attention_weights * enc_outputs, axis=1)
```

Luong (Multiplicative) Implementation

Simpler in its most common form (dot). No extra tanh layer is needed for scoring - just a similarity measure between vectors. The "general" variant adds one weight matrix; only the "concat" variant needs an extra tanh + vector, similar to Bahdanau.

$$\text{score}(s_t, h_j) = s_t^T \cdot h_j \quad (\text{dot variant, shown here})$$

```
# dot-product variant - simplest, no extra learned weights
score = matmul(s_t, enc_outputs, transpose_b=True)
attention_weights = softmax(score, axis=1)
context = sum(attention_weights * enc_outputs, axis=1)
h_tilde = tanh(Wc(concat([context, s_t]))) # attentional hidden state
```

6. Alignment (Score) Functions Compared

Alignment (Score) Functions

Name	Formula	Used By
Additive (Concat)	$v^T \tanh(W1.s + W2.h)$	Bahdanau (original)
Dot	$s^T . h$	Luong (simplest, needs equal dims)
General	$s^T . W . h$	Luong (adds a learned weight matrix)
Concat	$v^T \tanh(W.[s;h])$	Luong (alternate, similar to Bahdanau)

Figure 4: Bahdanau uses only the additive/concat formula; Luong offers three lighter-weight options.

A key contribution of Luong's paper was showing that you don't always need the extra tanh + vector layer to get good alignment scores - a simple dot product (when decoder and encoder hidden sizes match) or a single weight matrix ("general") often works just as well, with fewer parameters and faster computation.

7. Key Differences - Summary Table

Aspect	Bahdanau Attention	Luong Attention
Year / Paper	2014, Bahdanau et al.	2015, Luong et al.
Also known as	Additive attention	Multiplicative attention
Decoder state used	Previous state $s(t-1)$	Current state $s(t)$
When attention is applied	BEFORE the RNN step	AFTER the RNN step
Score function(s)	Additive (concat + tanh + v)	Dot, General, or Concat
Encoder type (original)	Bidirectional GRU	Stacked (multi-layer) LSTM
Parameter count	More (extra $W1, W2, v$)	Fewer, especially with dot variant

Context vector combined with	Input to the RNN cell	RNN's output hidden state
------------------------------	-----------------------	---------------------------

When to Use Which?

- **Bahdanau** tends to be preferred for tasks needing very precise alignment (its extra learned layer gives it more expressive scoring), and is a natural fit when using GRUs.
- **Luong (dot/general)** is faster and lighter, making it a popular default for LSTM-based decoders, especially when compute efficiency matters.
- In practice, both are largely historical stepping stones now - modern architectures use the **self-attention** introduced by the Transformer (see "Attention Is All You Need"), which generalizes and extends these same core ideas.

Both mechanisms proved the same core insight: letting a model dynamically decide what to focus on, rather than compressing everything into one fixed vector, was one of the most important ideas in the path toward today's Transformer-based language models.